

COLOR CLASH

BIL 482 – Design Patterns

Sprint Plan & Product / Sprint Backlog Document

Project Timeline: February 4, 2026 – March 29, 2026 | 3 Sprints

Team Members

Mehmet Yasin TOSUN • Mustafa GÖZÜTOK • Mehmet Umur ÖZÜ

1. Project Overview

Color Clash is a web-based two-dimensional fighting game developed as part of the TOBB ETU BIL 482 (Design Patterns) course. It is built from scratch using TypeScript and HTML5 Canvas with zero production dependencies. The primary goal of the project is to demonstrate how software design patterns (Singleton, Command, State, Factory, Strategy, Observer, Object Pool) are applied in practice within a game architecture.

Applied Design Patterns and Corresponding Classes:

- Singleton – GameEngine: single game loop and state orchestrator
- Command – MoveCommand, AttackCommand, SwitchModeCommand + InputHandler
- State – IdleState, MoveState, AttackState, HitState (ICharacterState)
- Factory – CharacterFactory: centralized player object creation
- Strategy – FireModeStrategy, WaterModeStrategy (IElementalMode)
- Observer – Subject / IObserver; GameHUD and HealthBar as subscribers
- Object Pool – ObjectPool<T>; Particle and Hitbox recycling

Technology Stack:

- TypeScript (strict mode) – type safety and OOP
- HTML5 Canvas 2D – rendering engine
- Vite – development server and production build
- GitHub – version control and team collaboration

2. Sprint Overview

The project was carried out in 3 sprints between February 4 and March 29, 2026. The actual sprint boundaries are determined based on the commit history. The Requirement → Design → Implementation → Test cycle was applied in every sprint.

Sprint	Date Range	Main Focus	Points	Status
Sprint 1	Feb 4 – Feb 28, 2026	Project Definition, SRS, Core Engine, Command, State, Factory, CollisionSystem	39	✓ Completed
Sprint 2	Feb 28 – Mar 27, 2026	Strategy (Fire/Water), GameConfig, CharacterState, Observer/UI (GameHUD, HealthBar), UC2 Combat Flow	34	✓ Completed
Sprint 3	Mar 27 – Mar 29, 2026	Particle, ParticleSystem, Object Pool, Observer Infrastructure, Performance, Final Documentation	27	✓ Completed

3. Commit History & Contribution Matrix

The table below shows the classes and components added by each team member, based on the GitHub commit history.

Commit Date	Developer	Added Classes / Changes
Feb 4, 2026	Mehmet Umur OZU	README.md (create & update), Project Definition & Plan Documents (#PA1-Part1) — commits: c7fd03f, 87c6f89, 54f2ded, aac132f
Feb 11, 2026	Mustafa GOZUTOK	SRS document uploaded (Add files via upload), old SRS deleted, updated version added — commits: 4bd44a8, f92c28c, ef4785d
Feb 28, 2026	Mustafa GOZUTOK	Character, CharacterFactory, MoveState, CollisionSystem — commit: 47ea7d9
Feb 28, 2026	Mehmet Umur OZU	Core module, Command pattern, IdleState (State pattern), InputHandler system — commit: 3e0ce2c
Feb 28, 2026	Mehmet Yasin TOSUN	UC1 & UC2 Implementation (Start Game Battle, Execute Combat Actions), moved documents to docs/ folder — commits: 1167e33, 5f02ac1
Mar 27, 2026	Mustafa GOZUTOK	Particle, ParticleSystem, Observer (Subject/IObserver), ObjectPool — commit: 9ef8b7a
Mar 27, 2026	Mehmet Umur OZU	GameConfig, CharacterState, WaterMode, TypeScript type definitions — commit: 72f06fb
Mar 27, 2026	Mehmet Yasin TOSUN	Strategy pattern (FireModeStrategy, WaterModeStrategy) and UI code (GameHUD, HealthBar) — commit: d77abcf

4. Product Backlog

All high-level requirements of the project are listed below. In line with the Agile approach, the backlog was kept at a high level initially; only the items for each sprint were detailed before that sprint began. Priority rankings and sprint assignments are based on the commit history.

ID	User Story / Requirement	Category	Priority	Points	Sprint 1	Sprint 2	Sprint 3
PB-01	Players should be able to move their character via keyboard (move left/right, jump); gravity and ground collision must work	Gameplay	High	8	✓ Done		
PB-02	Keyboard input should be translated into MoveCommand / AttackCommand / SwitchModeCommand via Command pattern; managed by InputHandler	Architecture / Command	High	8	✓ Done		
PB-03	Character state machine (IdleState, MoveState, AttackState, HitState) should be managed via State pattern; state constraints must be enforced	Architecture / State	High	8	✓ Done		
PB-04	Player characters (including positions, directions, and key bindings) should be created centrally via CharacterFactory	Architecture / Factory	High	5	✓ Done		
PB-05	AABB-based CollisionSystem should detect character-character and character-hitbox collisions; characters must not pass through each other	Physics / Collision	High	5	✓ Done		
PB-06	GameEngine should operate as a Singleton; a requestAnimationFrame-based 60 FPS game loop, rendering, and system synchronization must be provided	Architecture / Singleton	High	5	✓ Done		
PB-07	Players should be able to switch instantly between Fire (high damage / low defense) and Water (high defense / low damage) combat modes via Strategy pattern	Architecture / Strategy	High	8		✓ Done	
PB-08	UI components such as health bar and mode indicator should be updated in real-time via Observer pattern through GameHUD	Architecture / Observer + UI	High	8		✓ Done	
PB-09	Particle and Hitbox objects should be recycled via Object Pool pattern; 60 FPS stabilization and memory leak prevention must be ensured	Performance / Object Pool	Medium	5			✓ Done
PB-10	Mode-based visual effects (sparks, water drops, etc.)	Visual / Effects	Medium	5			✓ Done

ID	User Story / Requirement	Category	Priority	Points	Sprint 1	Sprint 2	Sprint 3
PB-11	should be rendered at impact moments via ParticleSystem	Use Case		8			
	UC1 (Start Game Battle) and UC2 (Execute Combat Actions) use case scenarios must be fully implemented		High			✓ Done	
PB-12	SRS (use cases, architecture, design pattern mapping) and all project documents must be completed	Documentation	Medium	5	✓ Partial	✓ Partial	✓ Done

5. Sprint 1 – Project Definition, SRS, Core Engine & Basic Mechanics

Date: February 4 – February 28, 2026 | **Sprint Points:** 39 points

5.1 Sprint Goal

By the end of this sprint, project documents (Project Definition, Project Plan, SRS) will be completed and added to the repository; GameEngine (Singleton), Command pattern (InputHandler, MoveCommand, AttackCommand, SwitchModeCommand), IdleState, Character, CharacterFactory, MoveState and CollisionSystem classes will be implemented, yielding a prototype in which two players can move independently and collision detection is functional.

5.2 Product Backlog Items for This Sprint

PB-01 (Movement), PB-02 (Command), PB-03 (State), PB-04 (Factory), PB-05 (CollisionSystem), PB-06 (GameEngine Singleton), PB-12 (Documentation – partial)

5.3 Requirement Phase

- Players should be able to move left/right and jump via keyboard; gravity and ground boundary must work (PB-01)
- Keyboard input must be separated into MoveCommand / AttackCommand / SwitchModeCommand via Command pattern (PB-02)
- Character state machine: IdleState, MoveState, AttackState, HitState; state constraints must be enforced (PB-03)
- Two player characters must be created with initial positions and key bindings via CharacterFactory (PB-04)
- AABB CollisionSystem: characters must not pass through each other; hitbox intersections must be detectable (PB-05)
- GameEngine must run as a single instance (Singleton); 60 FPS game loop and system synchronization must be provided (PB-06)

5.4 Design Phase

- Umur OZU: Overall layered architecture (Core / Systems / Entity / UI), README, GameEngine Singleton architecture, Command pattern interfaces, IdleState and InputHandler key map (P1: WASD+F+G, P2: Arrows+K+L)
- Mustafa GOZUTOK: Character class diagram (health, position, velocity, currentState, elementalMode), CharacterFactory creation flow, MoveState transition conditions, CollisionSystem AABB logic
- Yasin TOSUN: UC1 (Start Game Battle) and UC2 (Execute Combat Actions) use case flows and SRS contributions

5.5 Sprint Backlog

ID	Task	Assigned To	Type	Status	Commit / Note
S1-R1	Project definition and scope requirements were identified; Project Definition Document was completed	Umur OZU	Requirement	Done	Feb 4 commit: c7fd03f
S1-R2	Software requirements analysis was conducted; SRS document was created and uploaded to the repository	Mustafa GOZUTOK	Requirement	Done	Feb 11 commits: 4bd44a8, ef4785d

ID	Task	Assigned To	Type	Status	Commit / Note
S1-R3	Use case scenarios (UC1: Start Game Battle, UC2: Execute Combat Actions) were defined and added to the SRS	Yasin TOSUN	Requirement	Done	Feb 28 commit: 1167e33
S1-D1	Overall architecture design was completed (layered architecture: Core / Systems / Entity / UI); README prepared; GameEngine Singleton and Command pattern interfaces designed; InputHandler key map defined (P1: WASD+F+G, P2: Arrows+K+L)	Umur OZU	Design	Done	Feb 4 commits: 87c6f89, 54f2ded, aac132f
S1-D2	Design pattern research completed; Pattern-Use Case mapping decided (State, Command, Factory, Strategy, Observer, ObjectPool, Singleton)	Entire Team	Design	Done	Planning phase
S1-D3	Character, CharacterFactory, MoveState, CollisionSystem class designs completed; AABB collision architecture defined	Mustafa GOZUTOK	Design	Done	Feb 28 commit: 47ea7d9
S1-D4	UC1 (Start Game Battle) and UC2 (Execute Combat Actions) use case flow designs created	Yasin TOSUN	Design	Done	Feb 28 commit: 1167e33
S1-I1	README.md created and updated; project definition & plan documents added to repository (#PA1-Part1)	Umur OZU	Implementation	Done	Feb 4 commits: c7fd03f, 87c6f89, 54f2ded, aac132f
S1-I2	SRS document uploaded; old SRS deleted, updated version added	Mustafa GOZUTOK	Implementation	Done	Feb 11 commits: 4bd44a8, f92c28c, ef4785d
S1-I3	Character class (health, position, velocity, currentState, elementalMode), CharacterFactory, MoveState and CollisionSystem (AABB) implemented	Mustafa GOZUTOK	Implementation	Done	Feb 28 commit: 47ea7d9
S1-I4	GameEngine Singleton, Command pattern infrastructure (ICommand, MoveCommand, AttackCommand, SwitchModeCommand), IdleState and InputHandler system implemented	Umur OZU	Implementation	Done	Feb 28 commit: 3e0ce2c
S1-I5	UC1 (Start Game Battle) and UC2 (Execute Combat Actions) use case scenarios implemented; docs folder organized	Yasin TOSUN	Implementation	Done	Feb 28 commits: 1167e33, 5f02ac1
S1-T1	Movement, jumping and ground collision manually tested; independent control of two players verified; no key conflicts observed	Entire Team	Test	Done	
S1-T2	AABB collision detection and character overlap prevention tested and confirmed	Mustafa GOZUTOK	Test	Done	
S1-T3	Correct key mapping via Command pattern and dual-player management by InputHandler verified	Umur OZU	Test	Done	

5.6 Test Phase Summary

- Movement, jumping and ground collision manually tested
- Independent keyboard control of both players verified; no key conflicts observed
- AABB collision detection confirmed to affect the correct character; overlap prevention validated
- IdleState ↔ MoveState transitions verified to work correctly based on key state

5.7 Sprint Review & Retrospective

Completed:

- Project documents added to repository (Umur OZU – Feb 4; Mustafa GOZUTOK – Feb 11)
- Core engine infrastructure (Singleton, Command, IdleState, InputHandler) ready (Umur OZU – Feb 28)
- Character entity layer (Character, CharacterFactory, MoveState, CollisionSystem) ready (Mustafa GOZUTOK – Feb 28)
- UC1 and UC2 basic flows implemented (Yasin TOSUN – Feb 28)

Carried over to Sprint 2:

- Strategy pattern (Fire/Water modes) and GameConfig constants will be implemented in Sprint 2
- Observer-based UI (GameHUD, HealthBar) will be completed in Sprint 2

6. Sprint 2 – Strategy, GameConfig, CharacterState & Observer/UI

Date: February 28 – March 27, 2026 | **Sprint Points:** 34 points

6.1 Sprint Goal

By the end of this sprint, the Strategy pattern (FireModeStrategy, WaterModeStrategy) and GameConfig central constants class will be operational; CharacterState type definitions and WaterMode will be added to configure game balance; GameHUD and HealthBar will be connected via Observer pattern; and the full UC2 combat flow (AttackState → Hitbox → CollisionSystem → Strategy damage → HitState → Observer notification) will be completed.

6.2 Product Backlog Items for This Sprint

PB-07 (Strategy), PB-08 (Observer + UI), PB-11 (UC2 full flow), PB-12 (Documentation – partial)

6.3 Requirement Phase

- FireModeStrategy: attack x1.5 / defense x0.7; WaterModeStrategy: defense x1.5 / attack x0.7 (PB-07)
- Mode switch via G (P1) / L (P2) keys; switch must be blocked during HitState or AttackState (UC3)
- Observer: HealthBar smooth animation via HealthChangeEvent; mode indicator update via ModeChangeEvent (PB-08)
- GameConfig: GRAVITY, MAX_HEALTH, ATTACK_DURATION constants; CharacterState union type; WaterMode type

6.4 Design Phase

- Umur OZU: GameConfig constants class schema, CharacterState union type structure, WaterMode interface
- Yasin TOSUN: IElementalMode interface and FireModeStrategy / WaterModeStrategy concrete classes; IObserver<T> / ISubject<T> interfaces; GameHUD component layout (health bars top corners, mode indicator, center overlay); HealthBar smooth animation and damage flash design

6.5 Sprint Backlog

ID	Task	Assigned To	Type	Status	Commit / Note
S2-R1	Strategy pattern requirements detailed: Fire Mode (attack x1.5 / defense x0.7) and Water Mode (defense x1.5 / attack x0.7); damage multipliers defined	Entire Team	Requirement	Done	SRS §5.4 UC3 detailing
S2-R2	UI components to be updated via Observer (HealthBar, mode indicator) and events to be emitted (HealthChangeEvent, ModeChangeEvent) defined	Yasin TOSUN	Requirement	Done	SRS §8.4 Observer row
S2-R3	Requirements for GameConfig constants (GRAVITY, MAX_HEALTH, ATTACK_DURATION, etc.) and CharacterState type definitions identified	Umur OZU	Requirement	Done	Mar 27 commit: 72f06fb

ID	Task	Assigned To	Type	Status	Commit / Note
S2-D1	Strategy pattern interface designed: IElementalMode (FireModeStrategy, WaterModeStrategy); calculateAttack() / calculateDefense() methods defined	Yasin TOSUN	Design	Done	Mar 27 commit: d77abcf
S2-D2	Observer pattern interfaces designed: IObserver<T> / ISubject<T>; GameHUD and HealthBar designated as Observers	Yasin TOSUN	Design	Done	Mar 27 commit: d77abcf
S2-D3	GameConfig central constants class and CharacterState / WaterMode type definitions designed	Umur OZU	Design	Done	Mar 27 commit: 72f06fb
S2-D4	GameHUD (health bars, mode indicators, state overlays) and HealthBar UI component designs completed	Yasin TOSUN	Design	Done	Mar 27 commit: d77abcf
S2-I1	GameConfig (GRAVITY, MAX_HEALTH, ATTACK_DURATION constants), CharacterState union type, WaterMode and general TypeScript type definitions implemented	Umur OZU	Implementation	Done	Mar 27 commit: 72f06fb
S2-I2	Strategy pattern: FireModeStrategy and WaterModeStrategy (IElementalMode interface) implemented; mode-based damage/defense multipliers integrated	Yasin TOSUN	Implementation	Done	Mar 27 commit: d77abcf
S2-I3	UI code added: GameHUD (Observer, health bars, mode indicators, start/gameover overlay) and HealthBar (smooth animation, damage flash) implemented	Yasin TOSUN	Implementation	Done	Mar 27 commit: d77abcf
S2-I4	UC2 combat flow completed: AttackState → Hitbox creation → CollisionSystem → activeStrategy.attack() → takeDamage() → HitState → Observer notification	Yasin TOSUN	Implementation	Done	Feb 28 commit: 1167e33 (base UC2) + Mar 27
S2-T1	Damage calculations for FireMode (x1.5) and WaterMode defense multiplier (x1.5) tested and verified to produce expected results	Entire Team	Test	Done	
S2-T2	Observer notifications correctly triggering HealthBar animation and mode indicator update verified	Yasin TOSUN	Test	Done	
S2-T3	Mode switch prevention during AttackState and HitState verified; delta-time consistency at 60Hz and 144Hz confirmed	Umur OZU	Test	Done	

6.6 Test Phase Summary

- Fire mode damage multiplier (x1.5) and Water mode defense multiplier (x1.5) verified to produce expected results
- Observer notifications correctly triggering HealthBar animation and instant mode indicator update confirmed
- Mode switch prevention during AttackState and HitState tested and verified

- Game speed consistency at 60Hz and 144Hz monitors via delta-time confirmed

6.7 Sprint Review & Retrospective

Completed:

- Strategy pattern (Umur OZU – GameConfig + Yasin TOSUN – FireModeStrategy/WaterModeStrategy) – Mar 27
- Observer-based UI (Yasin TOSUN – GameHUD + HealthBar) – Mar 27
- Full combat flow (UC2) integration completed

Carried over to Sprint 3:

- Particle effects, ParticleSystem and Object Pool will be implemented in Sprint 3
- Final documentation review will be completed in Sprint 3

7. Sprint 3 – Particle, Object Pool, Observer Infrastructure & Final Submission

Date: March 27 – March 29, 2026 | **Sprint Points:** 27 points

7.1 Sprint Goal

By the end of this sprint, Particle and Hitbox objects will be recycled via Object Pool (ObjectPool<T>) pattern; mode-based visual effects (fire sparks / water drops) will be activated through ParticleSystem; the Observer infrastructure (Subject / IObserver) will be extended to cover Particle integration; and the project will be ready for submission.

7.2 Product Backlog Items for This Sprint

PB-09 (Object Pool), PB-10 (ParticleSystem), PB-12 (Documentation – completion)

7.3 Requirement Phase

- ObjectPool<T>: acquire() / release() methods; new object creation when pool is full (PB-09)
- Particle: position, velocity, color (mode-based), lifetime; ParticleSystem: spawn, update, render, lifetime management (PB-10)
- Observer pattern infrastructure: Character as Subject; GameHUD, HealthBar as IObserver; particle spawn event notification system
- Off-screen entity culling: particles outside the viewport must not be rendered
- Memory target: heap < 100 MB after 1000 particle spawn/destroy cycles; FPS >= 55

7.4 Design Phase

- Mustafa GOZUTOK: ObjectPool<T> generic structure (acquire/release/pool capacity), Particle data model (x, y, vx, vy, color, lifetime, active), ParticleSystem workflow (integration with ObjectPool, mode-based color assignment, culling), Subject/IObserver interfaces

7.5 Sprint Backlog

ID	Task	Assigned To	Type	Status	Commit / Note
S3-R1	Object types to be managed via Object Pool (Particle, Hitbox) identified; 60 FPS performance target and memory limits finalized	Mustafa GOZUTOK	Requirement	Done	SRS §6.2 Perf. + §11
S3-R2	ParticleSystem scope defined: mode-based visual effects (fire sparks / water drops), lifetime management and off-screen entity culling	Mustafa GOZUTOK	Requirement	Done	
S3-R3	Observer interface implementations (Subject, IObserver) requirements finalized to cover Particle and GameHUD integration	Mustafa GOZUTOK	Requirement	Done	Mar 27 commit: 9ef8b7a
S3-D1	ObjectPool<T> generic class designed; acquire() / release() methods and pool capacity defined	Mustafa GOZUTOK	Design	Done	Mar 27 commit: 9ef8b7a
S3-D2	Particle class (position, velocity, color, lifetime) and ParticleSystem	Mustafa GOZUTOK	Design	Done	Mar 27 commit: 9ef8b7a

ID	Task	Assigned To	Type	Status	Commit / Note
S3-D3	(ObjectPool management, spawn, update, render) designed	Mustafa GOZUTOK	Design	Done	Mar 27 commit: 9ef8b7a
	Subject / IObserver observer infrastructure and GameHUD integration points designed				
	Particle and ParticleSystem (Object Pool based, mode-specific effects: sparks / water drops / fire) implemented and integrated into GameEngine				
S3-I1	Observer pattern infrastructure (Subject, IObserver interfaces) implemented; Character connected as Subject, GameHUD and HealthBar as Observers	Mustafa GOZUTOK	Implementation	Done	Mar 27 commit: 9ef8b7a
S3-I2	Observer pattern infrastructure (Subject, IObserver interfaces) implemented; Character connected as Subject, GameHUD and HealthBar as Observers	Mustafa GOZUTOK	Implementation	Done	Mar 27 commit: 9ef8b7a
S3-I3	ObjectPool<T> generic class implemented; pool integration for Hitbox and Particle completed	Mustafa GOZUTOK	Implementation	Done	Mar 27 commit: 9ef8b7a
S3-I4	All SRS, SDD and architecture documents (including design pattern-use case mapping table) completed and moved to docs/ folder	Yasin TOSUN / Entire Team	Implementation	Done	Feb 28 commit: 5f02ac1; final review
S3-T1	No memory leak observed after 1000+ particle spawn/destroy cycles; heap stability confirmed	Mustafa GOZUTOK	Test	Done	
S3-T2	Correct object recycling verified in ObjectPool acquire/release cycles; FPS confirmed to stay >= 55	Mustafa GOZUTOK	Test	Done	
S3-T3	Particle effects confirmed to be correctly triggered on mode switch and impact via Observer notifications	Entire Team	Test	Done	
S3-T4	Browser compatibility (Chrome, Firefox, Edge) final check completed; production build via npm run build verified	Entire Team	Test	Done	Project ready for submission

7.6 Test Phase Summary

- No memory leak observed after 1000+ particle cycles; heap remained stable
- Object identities confirmed to be reused after ObjectPool acquire/release
- Fire mode spark and Water mode water drop effects verified to spawn at correct color and position
- Final build (npm run build) successfully completed on Chrome, Firefox and Edge; playability confirmed

7.7 Sprint Review & Retrospective

Completed:

- Particle, ParticleSystem, Observer infrastructure and ObjectPool – Mustafa GOZUTOK (Mar 27: 9ef8b7a)
- All SRS, SDD, architecture and pattern-use case mapping documents completed
- Project runs smoothly in browser with production build; ready for submission

Overall Assessment:

- Product Backlog was reviewed before each sprint; only the items for that sprint were detailed
- The Requirement → Design → Implementation → Test cycle was fully applied in every sprint
- Commit history directly reflects sprint boundaries and individual contributions

8. Conclusion

The Color Clash project was successfully completed across 3 sprints within the Agile sprint cycle framework. Seven different design patterns were applied in practice, a traceable contribution matrix was established via commit history, and the project was brought to submission-ready status.

Total Sprint Points: 100 | All Commits Are Based on Real GitHub History
